

(Lecture - 11)Time and Space Complexity

→ Time Complexity != Time taken

Time Complexity is defined as the time taken by the algorithm as our input size.

For same program time is different to run on different-different system.

* Asymptotic Notation:

```
for (int i = 0; i < N; i++) {
    sum += arr[i];
}
```

Algorithm: Worst case: Big-O:

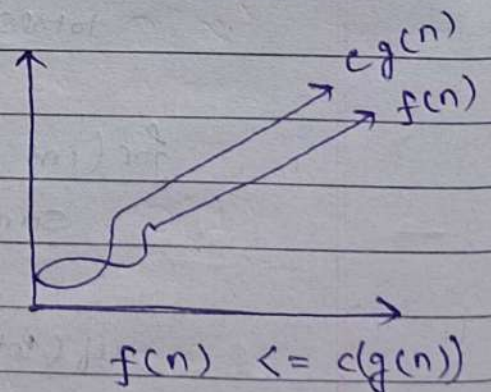
$$n > n_0$$

$$3x + 2 \leq C(n)$$

$$3x + 3 \leq Cn$$

$$C \geq 4$$

Worst $O(N^2)$



$$f(n) \leq c(g(n))$$

Best case : Big-omega (Ω)

$$f(n) \geq c(g(n))$$

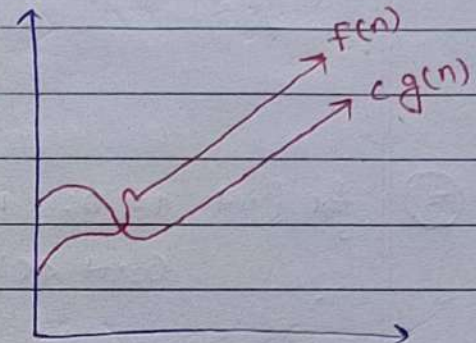
$$n \geq n_0$$

$$c > 0, n_0 \geq 1$$

$$f(n) = 3n + 2, g(n) = n$$

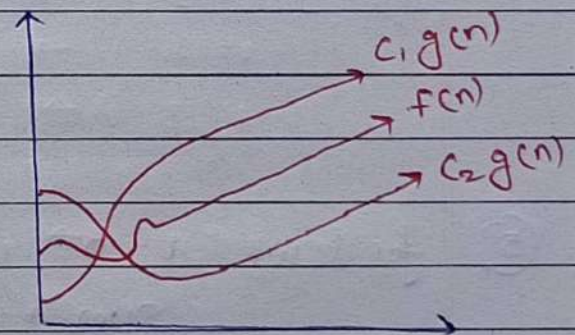
$$3n + 2 \geq c n$$

$$3n + 2 \geq \Omega$$



Average Case : Big theta (Θ)

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)$$



① Search an Element :

4 3 9 7 6 5 Input

```
for (int i = 0; i < n; i++) {
```

```
    if (arr[i] == input) {
```

```
        cout << i;
```

```
        break;
```

```
    }
```

```
} }
```

Best : $\Omega(1)$

Worst : $O(N)$

Average : $\Theta(N/2)$

② for (int i = 0; i < n; i++) {

cout << i << " ";

}

$O(N)$

$\Theta(N)$

$\Omega(N)$

③ for (int i = 0; i < n; i++) {

cout << "pari";

}

$O(N)$

$\Theta(N)$

$\Omega(N)$

④ for (int i = 0; i < n; i++) {

for (int j = 1; j <= 100; j++) {

cout << "Hey";

}

}

$100 \times N$

$= 100N$

$O(N)$

↳ neglect constant

⑤ for (int i = 0; i < n; i++) {

sum += i+1;

}

$O(N)$

⑥ sum = $n(n+1)/2$;

$O(1)$

⑦ for (int i = 1; i <= n; i++) {

for (int j = 1; j <= n; j++) {

cout << "DSA";

}

}

$O(N^2)$

($n+n+n+n+\dots+n$ times)

($n \times n$) = n^2

Teacher's Signature.....

⑧ for (int i = 1; i <= n; i++)
 for (int j = 1; j <= i; j++)
 cout << "placement";

For i = 1 → j = 1 to 1 = 1
 i = 2 → j = 1 to 2 = 2
 i = 3 → j = 1 to 3 = 3
 !
 i = n → j = 1 to n = n

$$(1 + 2 + 3 + 4 + \dots + n)$$

$$\frac{n^2 + n}{2} \Rightarrow O(n^2)$$

⑨ for (int i = 2; i <= n; i++) {
 for (int j = 1; j <= i^2; j++) {
 cout << "Rohit";
 }
 }

For i = 2 → j = 1 to 4 = 4
 i = 3 → j = 1 to 9 = 9
 i = 3 → j = 1 to 16 = 16
 !
 i = n → j = 1 to n^2 = n^2

$$(4 + 9 + 16 + \dots + n^2)$$

$$\Rightarrow O(n^2)$$

$$1^2 + 2^2 + 3^2 + 4^2 + \dots + n^2$$

$$\frac{n(n+1)(2n+1)}{6} \Rightarrow \frac{(n^2+n)(2n+1)}{6}$$

$$= \frac{2n^3 + n^2 + 2n^2 + n}{6}$$

$$\Rightarrow O(n^3)$$

⑩ for (int i = 1; i <= n; i++)
 for (int j = 1; j <= m; j++)
 cout << "Complexity";

For i = 1 → j = 1 to m = m
 i = 2 → j = 1 to m = m
 i = 3 → j = 1 to m = m
 !
 i = n → j = 1 to m = m

$$m + m + m + m + \dots + n \text{ times}$$

$$O(m \cdot n)$$

⑪ for (int i = 1; i <= n; i++)
 for (j = 1; j <= i^2; j++)
 for (k = 1; k <= n/2; k++)
 cout << "pari";

For $i=1 \rightarrow j=1$ to $1 \rightarrow k \rightarrow n/2$ time

For $i=2 \rightarrow j=1$ to $4 \rightarrow k \rightarrow 4 \times n/2$ times

For $i=3 \rightarrow j=1$ to $9 \rightarrow k \rightarrow 9 \times n/2$ times

For $i=n \rightarrow j=1$ to $n^2 \rightarrow k \rightarrow n^2 \times n/2$ times.

$$\underbrace{(1^2 + 2^2 + 3^2 + 4^2 + \dots + n^2)}_0 \times n/2$$

$$(\eta^3) \times n/2$$

$$= O(\eta^4)$$

(12) for (int $i=1$; $i < n$; $i = i * 2$)
cout << "logarithm";

$$O(\log n)$$

1	2	4	8	16	n
2^0	2^1	2^2	2^3	2^4	2^k

$k+1$ times

$$\eta = 2^k$$

$$k = \log_2 n$$

(13) $\text{for}(i = n/2; i \leq n; i++) \rightarrow n/2$
 $\text{for}(j = 1; j \leq n/2; j++) \rightarrow n/2$
 $\text{for}(k = 1; k \leq n; k++) \rightarrow n$

All are independent:

$$n \times n/2 \times n/2$$

$$O(n^3)$$

(14) $\text{for}(\text{int } i = n/2; i \leq n; i++) \rightarrow n/2$
 $\text{for}(j = 1; j \leq n; j++) \rightarrow n$
 $\text{for}(k = 1; k \leq n; k = k * 2) \rightarrow \log n$

$$\frac{n}{2} \times n \times \log n$$

$$\text{Ans: } O(n^2 \log n)$$

(15) Decimal to Binary:
 $O(\log n)$

(16) $\text{for}(\text{int } i = n/2; i \leq n; i++) \rightarrow n/2$
 $\text{for}(\text{int } j = 1; j \leq n; j = 2 * j) \rightarrow \log n$
 $\text{for}(\text{int } k = 1; k \leq n; k = 2 * k) \rightarrow \log n$

$$n/2 \times \log n \times \log n$$

$$O(n \log^2 n)$$

(17)

```
For (int i = 1; i <= n; i++)
```

```
    for (int j = 1; j <= n; j = i + 1)
```

```
        cout << "RAM";
```

For $i = 1$: $j = 1$ to $n \Rightarrow n$

For $i = 2$: $j = 1, 3, 5, 7, \dots, n \Rightarrow n/2$

For $i = 3$: $j = 1, 4, 7, 10, \dots, n \Rightarrow n/3$

For $i = n$: $j = 1, n \Rightarrow n/n$

$$n + n/2 + n/3 + \dots + n/n$$

$$n \left[1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n} \right]$$

$$n(\log_e n)$$

$$O(n \log n).$$

lecture - 12Time and Space Complexity

Time Complexity states that the amount of time taken by an algorithm to run as a function of length of the input.

Compare : $O(\log n)$ & $O(\sqrt{n})$

1.	$n = 10$	3.3
2.	$n = 100$	10
3.	$n = 1000$	100
4.	$n = 10,000$	1000
5.	$n = 10^6$	100000 = 10^5

$O(\log n)$ is best.

$$O(1) < O(\log n) < O(\sqrt{n}) < O(n) < O(n \log n)$$

$$< O(n^2) < O(n^3) < O(2^n) < O(n!)$$

$O(\log n)$:

↳	1	2	4	8	16	32	...	n
↳	n	$n/2$	$n/4$	$n/8$	-	-	-	1

Space Complexity states that amount of space taken by an algorithm to run a function of the length of input.

Auxiliary space : Not included input size

Total time Complexity : Include input size.

* input arr[n]

create an array of size n and copy the element.

Input :

20	10	25	15	27	17
----	----	----	----	----	----

ans :

20	10	25	15	27	17
----	----	----	----	----	----

Auxiliary space : $O(N)$

Total time taken : $N + N = 2N$
 $O(N)$

* `int arr = { 10, 20, 30, 40, 50};`

`int ans[5];`

`for (int i = 0; i < 5; i++) {`

`ans[i] = arr[i] * 10;`

`}`

T.C : $O(N)$

S.C : $O(N)$

`for (int i = 0; i < 5; i++) {`

`cout << ans[i] << " " << ;`

T.T.C : $O(N)$

`}`

Output : 100, 200, 300, 400, 500

→ For prime Number:

We have to check from 2 to $(N/2)$.

→ prime No: check until $\sqrt{n}$. (optimal)

* prime No:

if ($N < 2$)

return 0;

for (int $i = 2$; $i \leq \text{sqrt}(N)$; $i++$) {

if ($N \% i == 0$)

return 1;

}

* Insertion Sort:

for (int $i = 0$; $i < n - 1$; $i++$) {

for (int $j = i$; $j > 0$; $j--$) {

}

}

$i = 0$ $i = 1$ $i = 2$ $i = 3$... $i = n$

0 to 0 1 to 0 2 to 0 3 to 0 ... n to 0

$[0 + 1 + 2 + 3 + 4 + 5 + 6 + \dots + n]$

$$\frac{n(n+1)}{2} = \frac{n^2+n}{2} = O(n^2)$$

↓

Time Complexity

Space Complexity: $O(1)$

* Bubble sort :

```
for (int i = 0; i < n - 1; i++)
```

```
    for (int j = 0; j < n - 1 - i; j++)
```

$i = 0 \Rightarrow j = 0 \text{ to } n-1$

$i = 1 \Rightarrow j = 0 \text{ to } n-2$

$i = 2 \Rightarrow j = 0 \text{ to } n-3$

$i = 3 \Rightarrow j = 0 \text{ to } n-4$

$i = n \Rightarrow j = 0 \text{ to } 1$

$\{ 1 + 2 + 3 + 4 + \dots + n-3 + n-2 + n-1 \}$

$$\frac{n(n+1)}{2} \Rightarrow O(n^2)$$